

Capítulo 15

RI

Construindo um Modelo de Linguagem do Zero

Curso bilingue inspirado no LLM101n

Gerado em 07/08/2026

Capítulo 15 — Finetuning II: RL

Objetivo de aprendizagem: otimizar um modelo contra uma **pontuação** em vez de contra respostas prontas. Implementar policy gradient do zero, entender a penalidade de KL — e ver, medido, o que acontece quando a recompensa mede a coisa errada.

Pré-requisitos: Capítulos 12 (geração) e 14 (o modelo com formato de instrução).

Arquivos: - `recompensa.py` — duas recompensas, uma boa e uma **mal especificada de propósito** - `reinforce.py` — o policy gradient, com penalidade de KL - `experimento.py` — as três configurações que definem o capítulo - `exercicios.md` — exercícios

1. Quando não há resposta certa para imitar

O [Capítulo 14](#) ensinou por **imitação**: existiam respostas escritas, e o modelo aprendeu a reproduzi-las. A loss media o quanto ele acertava o texto de referência.

Muita coisa que se quer de um modelo não tem texto de referência. "Seja mais útil." "Não invente fatos." "Escreva no tom certo." Não existe a resposta correta para comparar — existem respostas melhores e piores.

Quando é assim, o que sobra é uma **pontuação**: o modelo gera, alguém (ou algo) avalia, e os gradientes empurram na direção do que pontuou bem.

É essa a diferença entre SFT e RL, e ela é de **fonte de sinal**, não de algoritmo. SFT tem um alvo; RL tem um juiz.

2. De onde vem a recompensa neste capítulo

Em RLHF de verdade, o juiz é um **modelo de recompensa** treinado em preferências humanas: mostram-se duas respostas a uma pessoa, ela escolhe, e um modelo aprende a prever a escolha.

Aqui não há gente para coletar preferências. Então usamos recompensas **calculáveis por programa** — e a escolha não é só de conveniência.

O [Capítulo 11](#) estabeleceu, medindo, quando dados gerados pelo próprio modelo ajudam: **quando existe um verificador externo**. Uma recompensa programática é exatamente isso — objetiva, reproduzível, e imune ao modelo se auto-elogiar.

É também o regime que mais cresce na prática: RL sobre matemática, código e tarefas com resposta verificável, onde ninguém precisa opinar porque dá para conferir.

3. O algoritmo cabe numa linha

O **REINFORCE** é o policy gradient mais simples que funciona:

```
loss = -(R - baseline) * log_prob(resposta_gerada)
```

Se a recompensa ficou **acima da média** do batch, a loss empurra para **aumentar** a probabilidade daquela resposta. Se ficou abaixo, para diminuir. É só isso.

O **baseline** — a média do batch — não muda o gradiente esperado; ele reduz a **variância**. Sem baseline, toda resposta com recompensa positiva é reforçada, inclusive as ruins, e o aprendizado fica lento e instável.

O PPO, usado no RLHF clássico, é o REINFORCE mais três coisas: um *critic* que estima o baseline, *clipping* para limitar o tamanho da atualização, e várias épocas por amostra. Complexidade que existe para estabilizar, não para mudar a ideia.

4. A penalidade de KL

Otimizar uma pontuação sem restrição leva o modelo para onde a pontuação for maior — e esse lugar pode não ter nada a ver com o que você queria. A defesa padrão é amarrar a política ao modelo de partida:

```
R_efetiva = R - β · log( P_política / P_referência )
```

O termo é a divergência de KL estimada na trajetória gerada. Quanto mais a política se afasta da referência, mais ela é penalizada.

O modelo de referência é o SFT do Capítulo 14, **congelado**. Ele é a memória do que o modelo era antes de começar a perseguir pontos.

5. Duas recompensas, e uma delas é ruim de propósito

Recompensa	O que se quer dizer	O que ela mede
comprimento	"responda com ~30 tokens"	a mesma coisa — não há atalho
pontos	"escreva frases bem pontuadas"	a fração de tokens que são pontuação

As duas coincidem em texto normal. É por isso que a segunda *parece* razoável quando você a escreve. Testando à mão, antes de qualquer treino:

Sequência	comprimento	pontos
30 tokens, sem pontos	1,00	0,00

30 tokens, metade pontos	1,00	0,50
30 tokens, só pontos	1,00	1,00

Uma resposta que é **só pontuação** tira nota máxima em "escreva frases bem pontuadas".

A recompensa não tem bug. Ela mede exatamente o que foi escrito. O problema é que o que foi escrito não é o que se queria — e é assim que toda recompensa mal especificada se parece **antes** de alguém otimizá-la: razoável.

6. O experimento

Três configurações, 400 passos cada. E **três** números por configuração, porque um só engana:

Configuração	Recompensa	KL	Loss em Machado
comprimento (boa), com freio	0,591 -> 0,743	2,75	4,1701
pontos (má), SEM freio	0,058 -> 0,996	14,87	4,8211
pontos (má), com freio	0,058 -> 0,958	11,63	4,3963
(partida, sem RL)	—	0,00	4,0784

E as respostas que saem:

```
comprimento, com freio : 'é a busão não me lembra-me para que um bom silencio.
                          Para que seriam da vida... De ficar corte...'
pontos, SEM freio : '.'
pontos, com freio : '.'
```

Como ler, e a ordem importa

A recompensa sobe nas três. Isso não é informação — é o que o algoritmo faz. Uma recompensa que *não* sobe indica bug, não sucesso. Reportar só essa coluna é a forma mais comum de fingir que um treino de RL funcionou.

A loss em Machado é o juiz, e o RL nunca a observa. Ela mede se o modelo continua sendo um modelo de português enquanto persegue pontos.

A KL diz o quanto a política se afastou. Ela é o mecanismo; a loss real é a consequência.

O que a segunda linha mostra

Recompensa **0,996** — praticamente perfeita. E o modelo responde `'.'` a qualquer coisa que você pergunte.

Ele não trapaceou. Ele **otimizou exatamente o que estava escrito**, e foi extremamente bem nisso. O erro foi meu, na especificação — e nenhuma quantidade de RL bem implementado conserta uma recompensa que mede a coisa errada.

Este é o argumento inteiro do capítulo, e vale além dele: **toda métrica é uma medida do que você quer, nunca o que você quer**. Enquanto ninguém otimiza a medida, a diferença não aparece. O RL é uma máquina de encontrar essa diferença.

7. O freio de KL não é um interruptor

Eu escrevi, antes de medir, que a terceira configuração mostraria o freio **segurando** o hacking.

Ela mostra o freio **perdendo mais devagar**. Com $\beta = 0,02$, a recompensa ainda chega a 0,958 e a resposta ainda é `'.'`. O que muda é o custo: +0,32 de loss em vez de +0,74 — o estrago cai pela metade, e é só.

Métrica	Sem freio	Com $\beta = 0,02$
recompensa final	0,996	0,958
KL	14,87	11,63
custo em português	+0,74	+0,32

A penalidade de KL **compra tempo e limita o dano**. Não substitui uma recompensa correta.

E a faixa útil é um ponto só

Varrendo o β com `e4_dial_do_kl.py`:

β	Recompensa	KL	Custo em português	Regime
0,00	0,058 -> 0,996	14,87	+0,743	hackeia por completo
0,02	0,058 -> 0,958	11,63	+0,318	hackeia quase igual
0,10	0,055 -> 0,218	1,41	+0,005	aprende sem custo
0,50	0,053 -> 0,064	0,31	+0,003	mal se move
2,00	0,053 -> 0,055	0,23	+0,002	congelado

Só $\beta = 0,10$ tem as duas coisas: recompensa subindo e custo indistinguível de zero. Um fator de cinco para baixo e o modelo hackeia; um fator de cinco para cima e ele paralisa.

E note a armadilha das duas últimas linhas: em $\beta = 0,5$ o custo é **menor** que em $\beta = 0,10$. Pela coluna do custo, seria a melhor configuração — mas a recompensa subiu 0,011 em 400 passos. O modelo não foi protegido; foi impedido de aprender.

Custo zero é o que você obtém não treinando. Uma métrica de segurança que melhora quando o sistema para de funcionar não está medindo segurança.

8. Uma observação sobre orçamento, que quase me custou o capítulo

A primeira versão deste experimento usava **150 passos** e **$lr = 1e-5$** . O resultado:

Orçamento	Recompensa	KL	Loss real
pontos, sem freio (150 passos)	0,052 -> 0,163	2,02	4,1229
pontos, sem freio (400 passos)	0,058 -> 0,996	14,87	4,8211

Com o orçamento curto, o texto ainda parecia português e o efeito parecia sutil. Eu quase publiquei aquela tabela como "demonstração de reward hacking". **Não era** — era um empurrãozinho na direção certa, e teria dado ao leitor a impressão errada de que o fenômeno é ameno.

É o terceiro caso deste curso com a mesma forma. No [Capítulo 11](#), orçamento curto **inverteu** a resposta de dois exercícios. No [Capítulo 14](#), uma learning rate pequena demais fez parecer que o SFT não instalava o comportamento.

Quando um fenômeno não aparece, a primeira pergunta não é "será que ele não existe?". É "eu dei orçamento para ele aparecer?".

9. E o DPO?

O RLHF clássico tem três estágios: SFT -> treinar um modelo de recompensa -> otimizar com PPO. É caro e instável.

O **DPO** (*Direct Preference Optimization*) mostra que, para o caso de preferências pareadas, dá para pular o modelo de recompensa e o RL: existe uma loss supervisionada que tem o mesmo ótimo. Você treina direto nos pares "esta resposta é melhor que aquela", com algo que se parece com uma cross-entropy.

Por que o capítulo não usa DPO, então? Porque **ele precisa de pares de preferência**, e aqui não temos. Com recompensa programática — o regime de matemática, código e tarefas verificáveis — o policy gradient continua sendo o caminho, e é o que a prática atual usa.

E a lição da Seção 6 não muda com o algoritmo: DPO otimiza a preferência que você deu, com a mesma obediência literal.

10. Resumo do capítulo

- RL entra quando **não há resposta certa para imitar** — só respostas melhores e piores.
- O **REINFORCE** é uma linha: $-(R - \text{baseline}) * \log_prob$. O baseline reduz variância.
- Recompensa **verificável por programa** é o regime honesto quando não há gente para opinar — e o que mais cresce na prática.
- **Reportar só a recompensa não prova nada**. Ela sempre sobe. É preciso uma métrica que o RL não observe.
- **Reward hacking não é trapaça**: é o modelo otimizando exatamente o que foi escrito. O erro está na especificação.
- A **penalidade de KL é um dial**, não um interruptor. Limita o dano; não conserta a recompensa.
- Se um fenômeno não aparece, **confira o orçamento antes de concluir que ele não existe**.

Próximo capítulo

Capítulo 16 — Deployment. O modelo está treinado, afinado e alinhado. Falta a parte que separa um experimento de um produto: servi-lo para mais de uma pessoa ao mesmo tempo, sem que a latência exploda.

Exercícios — Capítulo 15 (RL)

Faça na ordem. Soluções comentadas em [solucoes/](#) — **tente antes de olhar**.

Este capítulo usa o modelo do Capítulo 14. Se `../14-sft/modelo_sft.pt` não existir, rode antes `cd ../14-sft && python preparar_sft.py && python sft.py`.

[tempo] Os experimentos de RL são **lentos**: cada configuração de 400 passos leva ~50 min, porque cada passo gera 48 tokens com dois modelos (política e referência). Planeje.

E1 — SFT e RL, lado a lado (aquecimento)

Sem rodar: 1. O Capítulo 14 treinou com `cross_entropy` contra respostas prontas. Aqui a loss é `-(R - baseline) * log_prob`. Descreva, em uma frase cada, **de onde vem o sinal** nos dois casos. 2. Por que o RL precisa **gerar** durante o treino, e o SFT não? O que isso custa? 3. O baseline é a média da recompensa no batch. Ele não muda o gradiente esperado — só a variância. Explique por quê. (Dica: qual é o valor esperado de `R - média(R)`?)

E2 — Escreva uma recompensa ruim (importante)

O capítulo usa duas: `comprimento` (bem especificada) e `pontos` (mal especificada). 1. Escreva uma **terceira** recompensa que pareça razoável e seja hackeável. Antes de rodar, descreva **como** você espera que o modelo a hackeie. 2. Rode com `beta=0.0` e 400 passos. O modelo hackeou do jeito que você previu, ou achou outro caminho? 3. Toda recompensa da Seção 5 da apostila tem a forma "medir X como proxy de Y". Para a sua, qual é o X e qual é o Y — e onde eles deixam de coincidir?

E3 — O baseline importa mesmo? (importante)

O `reinforce.py` aceita `--sem-baseline`. 1. Rode a recompensa `comprimento` com e sem baseline, mesma semente. Compare a curva de recompensa e a variância dela entre passos. 2. Sem baseline, **toda** resposta com recompensa positiva é reforçada — inclusive as ruins. Explique por que isso não impede o aprendizado, só o torna mais lento. 3. Se a sua recompensa fosse sempre negativa (digamos, de -1 a 0), o que aconteceria sem baseline? E isso sugere o quê sobre normalizar recompensas?

E4 — Onde fica a faixa útil do β

Rode `solucoes/e4_dial_do_kl.py`, que varre o β . 1. Faça a tabela de β contra três colunas: recompensa final, KL e **custo em português**. Onde está a faixa em que a recompensa sobe e o custo é pequeno? 2. Entre $\beta = 0,02$ e $\beta = 0,10$ há um fator de cinco. O que muda no comportamento do modelo nessa faixa? Por que a transição é tão abrupta? 3. Com β grande o custo vai a zero. Isso é sucesso? Olhe também a coluna da recompensa antes de responder.

E5 — A recompensa não é o objetivo

1. Para as três configurações do `experimento.py`, ordene os modelos por **recompensa** e depois por **loss em Machado**. As ordens coincidem?
2. Imagine que você só reportasse a recompensa num relatório. Que conclusão o leitor tiraria? E ela seria falsa em que sentido exatamente?
3. Proponha **duas** métricas que um projeto real de RLHF deveria acompanhar além da recompensa, e diga o que cada uma detectaria que a recompensa não detecta.

E6 — Conserte a recompensa

O problema de `pontos` não é o β — é a especificação. 1. Reescreva `muitos_pontos` para capturar melhor a intenção original ("escreva frases completas e bem pontuadas"). Pense no que distingue pontuação **útil** de pontuação **repetida**. 2. Rode com `beta=0.0`. A sua versão resiste sem freio nenhum? 3. Se resistiu: existe alguma recompensa que resista a qualquer otimização? Se não resistiu: o que isso diz sobre a estratégia de "escrever uma recompensa melhor"?

E7 — DPO (desafio)

O DPO troca o RL por uma loss supervisionada sobre **pares** de preferência. 1. Gere pares com o próprio modelo: para cada pedido, amostre duas respostas e rotule a de maior recompensa como preferida. Você acabou de construir um dataset de preferências sintético. 2. Implemente a loss do DPO e treine. Compare com o REINFORCE em recompensa, KL e custo. 3. **A pergunta que importa:** o DPO evita o reward hacking? Justifique pelo que ele otimiza — e note que os seus pares foram rotulados pela **mesma** recompensa ruim.